macOS Identity-Threat & Account-Lockout Hardening (Password Spray, Smart Lockout, SecureToken)

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary — macOS Identity-Threat & Account-Lockout Hardening 4

What you get . . . 4

What this delivers . . . 4

The standard deployment model . . . 4

Outcome . . . 4

1. Solution Overview — macOS Identity-Threat & Account-Lockout Hardening 5

What this solution is . . . 5

How a Mac ends up attacking your identity (the mechanics) . . . 5

The hardening model . . . 6

What you can control . . . 6

Limitations (be honest with buyers) . . . 6

The standard deployment model . . . 7

2. Requirements Guide — macOS Identity-Threat & Account-Lockout Hardening 7

2.1 Entra sign-in logs (and, where licensed, Identity Protection) . . . 7

2.2 Smart Lockout configuration access . . . 7

2.3 Conditional Access for MFA enforcement . . . 8

2.4 Intune with enrolled macOS devices . . . 8

2.5 A Mac pilot scope group . . . 8

2.6 Hybrid vs cloud-only lockout model confirmed . . . 8

2.7 macOS credential / keychain / SecureToken remediation authority . . . 8

2.8 Microsoft Graph PowerShell for discovery and reporting . . . 9

Checklist . . . 9

3. Implementation Guide — macOS Identity-Threat & Account-Lockout Hardening 9

Step 1 — Create the scope groups . . . 9

Step 2 — Assess: identify Mac-originated lockout / spray sources . . . 10

Step 3 — Configure Smart Lockout . . . 10

Step 4 — Enforce MFA (Conditional Access), piloted . . . 10

Step 5 — Prepare the Mac-side remediation . . . 11

Step 6 — Pilot the remediation on the affected Macs . . . 11

Step 7 — Validate the reduction . . . 11

Step 8 — Expand and set the review cadence . . . 12

5. Testing Guide — macOS Identity-Threat & Account-Lockout Hardening 12

Test 1 — Mac-originated failures identified . . . 12

Test 2 — Smart Lockout configured . . . 12

Test 3 — MFA enforced for the pilot . . . 12

Test 4 — Mac credential state remediated . . . 12
Test 5 — Failures reduced . . . 13
Test 6 — No new lockouts / spray detections for the pilot . . . 13
Test 7 — Evidence captured . . . 13
Pass/fail gate . . . 13
6. Troubleshooting Guide — macOS Identity-Threat & Account-Lockout Hardening 13
A. Failures keep coming from a Mac after the password change . . . 13
B. Account keeps locking out . . . 14
C. Password-spray / risk detection still firing . . . 14
D. MFA policy problems . . . 14
E. SecureToken / local-account divergence . . . 15
F. Scope / targeting problems . . . 15
G. Graph / permissions problems . . . 15
H. When to escalate . . . 15
7. Best Practices — Identity protection on Macs 15
8. Deliverables Summary — macOS Identity-Threat & Account-Lockout Hardening 16
Documentation . . . 16
PowerShell scripts . . . 17
Named objects (reused verbatim) . . . 17
What the buyer receives . . . 17
Outcome . . . 18
Appendix A — PowerShell Scripts 19

Executive Summary — macOS Identity-Threat & Account-Lockout Hardening

White-label · Security

Macs are a quiet source of identity trouble. When a user's Entra password changes, the Mac's keychain, mail profile, and Wi-Fi often keep re-submitting the **old** password automatically — a steady stream of **failed sign-ins** that Entra reads as a **password-spray** attack and that triggers **account lockouts** blocking the legitimate user. Sometimes it is tangled up with SecureToken and local-account state that has drifted out of sync with the cloud. This solution hardens the **identity posture of the Mac fleet**: it finds the Mac-originated failures, enforces the tenant controls that stop them from locking real users out, and cleans up the Mac-side credential state that causes the failures in the first place — under Zero-Trust and least-privilege.

This is a complete, deploy-it-today package written for an administrator with no prior identity-hardening experience.

What you get

• A plain-language Solution Overview of how macOS authentication, saved credentials, keychain, and

SecureToken contribute to failed sign-ins, lockouts, and password-spray noise — and the hardening model that fixes it.

• A Requirements Guide with verify / implement-if-missing for every prerequisite (Entra sign-in logs /

Identity Protection, Smart Lockout, MFA via Conditional Access, Intune, a pilot).

• An Implementation Guide of precise, ordered steps: identify Mac-originated lockout/spray sources from

the sign-in logs, enforce Smart Lockout + MFA, remediate stale Mac credentials/keychain/SecureToken, validate the reduction, then expand.

• Five PowerShell scripts for sign-in-failure/lockout discovery by device/OS, Smart-Lockout-and-MFA

validation, a before/after exposure report, safe remediation, and a Mac scope-group builder.

• Testing and Troubleshooting guides plus Best Practices for identity protection on Macs.

What this delivers

A Mac fleet that stops driving account lockouts and password-spray detections: the sources identified from Entra sign-in logs, Smart Lockout and MFA enforced so a burst of Mac failures no longer locks a real user out, and the stale Mac credential state cleaned up — proven on a pilot before you expand.

The standard deployment model

Assess → Configure → Pilot → Validate → Expand.

Outcome

Fewer lockouts, less spray noise, and a hardened Mac identity posture — deployable by a single administrator and repeatable on a review cadence.

1. Solution Overview — macOS Identity-Threat & Account-Lockout Hardening

What this solution is

This solution hardens the **identity posture of the Mac fleet** so macOS clients stop generating failed sign-ins against Microsoft Entra ID — the failed sign-ins that feed **password-spray** detections and trigger **account lockouts** that block legitimate users. It does three things in one program: it **identifies** which lockouts and spray-detection noise actually originate from Macs (read from Entra sign-in logs), it **hardens** the tenant with Entra **Smart Lockout** and enforced **MFA** so a burst of Mac-driven failures no longer locks a real user out, and it **remediates** the Mac-side credential and keychain state that causes the same account to keep failing — all under Zero-Trust and least-privilege.

The object being hardened here is the **identity surface as the Mac fleet touches it**: Entra sign-in behavior, the lockout/MFA policy that governs it, and the stale macOS credential state that keeps re-submitting a wrong or expired password. It is deliberately distinct from certificate or VPN authentication work (a separate solution) — here the problem is *bad repeated sign-ins from Macs* and the lockout/spray fallout they cause.

How a Mac ends up attacking your identity (the mechanics)

A Mac rarely "attacks" on purpose. The failed-sign-in storm almost always comes from **saved or stale credentials** that macOS keeps re-presenting on the user's behalf:

- **Keychain-stored passwords.** macOS Keychain stores the passwords for Wi-Fi/802.1X, mail

(Exchange/Outlook profiles), calendar, and app accounts. When the user's **Entra password changes** (or expires) but the keychain still holds the **old** password, macOS — and the apps that read from it — keep retrying the old password automatically, in the background, on a schedule. Each retry is a **failed sign-in** in Entra. Enough of them, fast enough, and Entra's protection logic reads it as a **password-spray / brute-force pattern** and/or **locks the account**.

- **Cached / saved sign-ins in mail and browser profiles.** A macOS Mail account, an Outlook profile,

or a browser-saved credential that still holds the pre-change password behaves the same way: silent, repeated, automatic failed sign-ins.

- **Stale device or WPJ (Workplace Join) state.** A Mac whose registered/joined identity state is

stale can repeatedly present old tokens or trigger repeated interactive-auth failures.

- **SecureToken / local-account divergence.** On macOS, the local account password and its

SecureToken (the FileVault/secure-enclave credential grant) can drift out of sync with the cloud (Entra) password after a reset. When they diverge, the user is prompted repeatedly, keychain items become invalid, and the resulting re-auth attempts add to the failed-sign-in count. (This is the "tangled with SecureToken/keychain" case — the local secure state is a *contributor* to the cloud failed-sign-ins, and cleaning it up is part of remediation.)

The result is the same regardless of the exact source: **one or more Macs quietly generate a stream of failed sign-ins for a user**, and Entra — doing its job — either flags the user in password-spray / risk detections or locks the account, so a legitimate person can't sign in.

The hardening model

1. **Identify the sources.** Read Entra **sign-in logs** (interactive and non-interactive) and filter

for **failures** whose **operating system is macOS** (and the failure/error codes that map to bad password, account locked, and MFA-required). This tells you *which users, which Macs, and which apps* are producing the failures — recorded in the findings object.

1. **Harden the tenant.** Turn on / tune Entra **Smart Lockout** (lockout threshold and duration) so

a burst of bad-password attempts locks *the attacker's view*, not the legitimate user, and enforce **MFA** (via Conditional Access) so a correct-password-only attempt is never sufficient. This raises the cost of spray and shrinks the lockout blast radius on real users.

1. **Remediate the Mac side.** Clear the stale keychain items, re-sync the saved credential in mail /

Wi-Fi / app profiles, and resolve any SecureToken / local-account divergence so the Mac stops re-submitting the wrong password. This removes the *cause*, not just the symptom.

1. **Validate the reduction.** Re-read the sign-in logs after remediation and confirm the

Mac-originated failures (and the lockouts/spray detections they drove) have dropped for the pilot population — then expand across the fleet.

What you can control

• **Visibility:** which Mac-originated failures, users, apps, and error codes are happening (from

Entra sign-in logs / Identity Protection).

• **Smart Lockout:** the lockout threshold and duration in the Entra authentication-methods /

password-protection settings.

• **MFA enforcement:** a Conditional Access policy that requires MFA for the scoped population.

• **Mac credential state:** keychain hygiene, saved-credential re-sync, and SecureToken / local-account

alignment on the endpoint (guided; the endpoint fixes are performed on the Mac, and Intune is used to scope and deliver the guidance/scripts).

• **Scope and cadence:** pilot group first, then broad; re-validate on a schedule.

Limitations (be honest with buyers)

• This reduces **Mac-originated** failed sign-ins, lockouts, and spray noise. Failures from other

platforms, real external attackers, or misconfigured line-of-business apps are out of scope (though the same sign-in-log method surfaces them).

• **Smart Lockout is tenant-level** for the cloud-only lockout behavior; some thresholds and the

on-premises lockout interplay depend on your environment (hybrid vs cloud-only). Confirm your model before tuning — flagged in Requirements.

- **Identity Protection risk detections** (the password-spray / risk signals) and risk-based

Conditional Access depend on your Entra ID licensing (P2 for the full risk-detection surface); Smart Lockout and MFA via Conditional Access are broadly available — verify per Requirements.

- macOS credential/keychain and **SecureToken** remediation is performed **on the endpoint**; the

scripts in this package read Entra sign-in data via Microsoft Graph and scope the work — they do **not** silently rewrite a user's keychain or SecureToken remotely. Endpoint steps are guided and gated by change control.

- This does not replace a broader identity-protection program; it targets the specific Mac-driven

lockout/spray problem.

The standard deployment model

> *Assess the Mac-originated failures from the sign-in logs, configure Smart Lockout + MFA and prepare the Mac-side remediation, pilot against a Mac scope group, validate the reduction, then expand — re-validating on a cadence. Repeatable pattern:* ***Assess*** *→* ***Configure*** *→* ***Pilot*** *→* ***Validate*** *→* ***Expand.***

2. Requirements Guide — macOS Identity-Threat & Account-Lockout Hardening

Verify every requirement before configuring. Each entry: Description · Why · Verify · Implement-if-missing. Category-wide prerequisites are in `global-requirements-security.md`.

2.1 Entra sign-in logs (and, where licensed, Identity Protection)

Description: Access to Microsoft Entra **sign-in logs** (interactive and non-interactive) and, if licensed, **Identity Protection** risk detections. **Why:** The whole method starts by reading which failed sign-ins come from Macs; the sign-in logs (with the operating-system and failure-reason fields) are the source of truth, and Identity Protection surfaces the password-spray / risk detections those failures drive. **Verify:** `https://entra.microsoft.com` to **Identity** → **Monitoring & health** → **Sign-in logs** loads and shows sign-ins; for risk detections, **Protection** → **Identity Protection** → **Risk detections** loads (if licensed). Run `MACLOCK-Discovery.ps1` to read Mac-originated failures via Microsoft Graph. **Implement:** Ensure an Entra plan that surfaces sign-in logs (all Entra tenants) and, for the full risk-detection surface and risk-based Conditional Access, **Entra ID P2** (flag this if you only need Smart Lockout + MFA, which do not require P2).

2.2 Smart Lockout configuration access

Description: Access to the Entra **Smart Lockout** settings (lockout threshold and duration). **Why:** Smart Lockout is the control that stops a burst of bad-password attempts from locking the legitimate user while still blocking the attacker's view — the core tenant-side hardening here. **Verify:** `https://entra.microsoft.com` to **Protection** → **Authentication methods** → **Password protection** shows **Lockout threshold** and **Lockout duration in seconds**. **Implement:** Confirm you hold a role that

can edit password protection (Security Administrator); record the current threshold/duration before changing them.

2.3 Conditional Access for MFA enforcement

Description: The ability to create a **Conditional Access** policy that requires MFA for the scoped population. **Why:** Enforcing MFA means a correct-password-only attempt (including a lucky spray hit) is never sufficient to sign in — it shrinks the value of the failed-sign-in noise and protects the account. **Verify:** `https://entra.microsoft.com` to **Protection** → **Conditional Access** → **Policies** loads and you can create a policy. **Implement:** Ensure Conditional Access is available (Entra ID P1/P2) and you hold Conditional Access Administrator or Security Administrator; if only Security Defaults is on, plan the move to a scoped Conditional Access MFA policy.

2.4 Intune with enrolled macOS devices

Description: Microsoft Intune with the target **Macs enrolled** and manageable. **Why:** Intune scopes and delivers the Mac-side remediation guidance/scripts to the right devices, and lets you confirm which Macs are managed and reporting. **Verify:** `https://intune.microsoft.com` to **Devices** → **All devices**; the target Macs appear with **OS = macOS** and are enrolled/compliant. **Implement:** Enrol the Macs (Automated Device Enrollment / Company Portal) and confirm they check in — see `global-requirements-security.md` G4.

2.5 A Mac pilot scope group

Description: A Microsoft Entra security group scoping the **pilot Mac population** (users and/or their Macs) for the scoped changes. **Why:** Everything that touches sign-in — MFA enforcement especially — is piloted before fleet rollout so a misconfiguration can't lock out the whole fleet. **Verify:** Entra to **Groups**; the pilot group exists — run `MACLOCK-Set-ScopeTargeting.ps1`. **Implement:** Create the scope groups with `MACLOCK-Set-ScopeTargeting.ps1` (or Entra to Groups to New group to Security).

2.6 Hybrid vs cloud-only lockout model confirmed

Description: A documented understanding of whether accounts are **cloud-only** or **hybrid** (synced from Active Directory), because lockout behaviour differs. **Why:** For hybrid accounts, an on-premises AD lockout policy and password writeback interact with Entra Smart Lockout; tuning cloud Smart Lockout without knowing the on-prem policy can produce surprising results. **Verify:** Entra to **Connect / Connect Sync** (or ask the directory owner) whether the affected users are synced; check whether an on-prem account-lockout GPO exists. **Implement:** Record the model in the register; if hybrid, coordinate the cloud Smart Lockout tuning with the on-prem lockout policy owner as part of change control (flag any uncertainty rather than guessing).

2.7 macOS credential / keychain / SecureToken remediation authority

Description: An agreed, change-controlled way to perform the **Mac-side** fixes: clear stale keychain items, re-sync saved credentials, and resolve SecureToken / local-account divergence. **Why:** The *cause* of the repeated failures lives on the Mac; fixing it requires acting on the endpoint (guided steps and/or Intune-delivered scripts), which is authentication-impacting and must be gated. **Verify:** A change ticket exists naming the pilot Macs, the remediation steps, and the rollback; you can deliver a script/config to the

pilot Macs via Intune. **Implement:** Adopt the change-control process from `global-requirements-security.md` G8; scope the remediation to `MACLOCK Scope - Pilot - Macs` first.

2.8 Microsoft Graph PowerShell for discovery and reporting

Description: The Microsoft Graph PowerShell SDK on the operator workstation, with consent to the read scopes the scripts use. **Why:** The discovery, validation, and report scripts read Entra **sign-in logs**, the current Smart Lockout / authentication-methods policy, and Conditional Access policies via Microsoft Graph. **Verify:** `Get-Module -ListAvailable Microsoft.Graph.Authentication` returns a version; run `MACLOCK-Discovery.ps1`. **Implement:** `Install-Module Microsoft.Graph -Scope CurrentUser`; consent to the read scopes (`AuditLog.Read.All`, `Directory.Read.All`, `Policy.Read.All`, `IdentityRiskEvent.Read.All`) on first connect.

Checklist

- Entra sign-in logs (and Identity Protection, if P2) — `MACLOCK-Discovery.ps1`
- Smart Lockout settings access (threshold + duration)
- Conditional Access available for MFA enforcement
- Intune with enrolled macOS devices
- Mac pilot scope group — `MACLOCK-Set-ScopeTargeting.ps1`
- Hybrid vs cloud-only lockout model confirmed
- macOS credential / keychain / SecureToken remediation authority + change control
- Microsoft Graph PowerShell installed — `MACLOCK-Discovery.ps1`

3. Implementation Guide — macOS Identity-Threat & Account-Lockout Hardening

Follow these steps in order, across Microsoft Entra (`https://entra.microsoft.com`) and Microsoft Intune (`https://intune.microsoft.com`). The steps produce and reuse three named objects verbatim: the findings register (`MACLOCK Sign-in Failure Findings - Mac-Originated - Sources`), the hardening register (`MACLOCK Lockout & Spray Register - Mac Fleet - Baseline`), and the remediation evidence pack (`MACLOCK Remediation Evidence Pack - Mac Credential State`). Scoped changes target the scope groups `MACLOCK Scope - Pilot - Macs` and `MACLOCK Scope - Broad - Macs`.

> ***Model:*** *Assess the Mac-originated failures, configure Smart Lockout + MFA and prepare the Mac-side remediation, pilot against the Mac scope group, validate the reduction, then expand.* ***Assess*** → ***Configure*** → ***Pilot*** → ***Validate*** → ***Expand.***

Step 1 — Create the scope groups

1. Run `MACLOCK-Set-ScopeTargeting.ps1 -Apply` to create the scope groups (`MACLOCK Scope - Pilot - Macs and MACLOCK Scope - Broad - Macs`), or create them in Entra to **Groups** → **New group** → **Security**.

1. Add a small, representative pilot population (the users whose Macs are producing failures) to `MACLOCK Scope - Pilot - Macs`.

Step 2 — Assess: identify Mac-originated lockout / spray sources

1. In Entra, go to **Identity** → **Monitoring & health** → **Sign-in logs**. Add filters: **Status = Failure**, and add the Operating system column (filter to macOS). Note the top users, the applications (e.g. Exchange/Outlook, Wi-Fi/802.1X, browser), and the failure reason / error codes** (bad password, account locked, MFA required).

1. If licensed, go to **Protection** → **Identity Protection** → **Risk detections** and note any **password-spray** or related detections and the users they hit.

1. Run `MACLOCK-Discovery.ps1` to read the same data via Microsoft Graph: it lists Mac-originated **failed sign-ins** grouped by user, app, and error code, and pulls the current **Smart Lockout** and **Conditional Access / MFA** posture.

1. Record every finding — user, Mac (device), app, error code, count — in `MACLOCK Sign-in Failure Findings - Mac-Originated - Sources, and record the current Smart Lockout threshold/duration and MFA posture in MACLOCK Lockout & Spray Register - Mac Fleet - Baseline` with their desired values.

Step 3 — Configure Smart Lockout

1. In Entra, go to **Protection** → **Authentication methods** → **Password protection**.

2. Record the current **Lockout threshold** and **Lockout duration in seconds**, then set the desired values from the register (a threshold that tolerates a small burst of Mac retries without locking the real user, and a duration appropriate to your environment). Smart Lockout locks out the attacker's view while recognizing the legitimate user.

1. If accounts are **hybrid** (Requirement 2.6), coordinate with the on-premises AD lockout policy owner before changing thresholds so cloud and on-prem lockout don't conflict.

1. Capture the resulting settings into `MACLOCK Remediation Evidence Pack - Mac Credential State`.

Step 4 — Enforce MFA (Conditional Access), piloted

1. In Entra, go to **Protection** → **Conditional Access** → **Policies** → **New policy**.

2. Name it `MACLOCK CA - Require MFA - Mac Pilot`. **Users:** assign `MACLOCK Scope - Pilot - Macs`.

Target resources: All cloud apps (or the specific apps producing failures). **Grant:** Require multifactor authentication. Start in **Report-only**, review the sign-in log's Conditional Access evaluation, then set to **On**.

1. (If P2) Consider a companion risk-based policy that requires MFA / password change on detected

sign-in risk for the same scope — name it `MACLOCK CA - Risk-Based MFA - Mac Pilot`.

1. Capture the policy settings into the evidence pack.

Step 5 — Prepare the Mac-side remediation

1. Build the remediation guidance for the pilot Macs, covering: clearing **stale keychain items** for

the affected accounts (Wi-Fi/802.1X, mail, app passwords); **re-syncing** the saved credential in the macOS Mail / Outlook profile and Wi-Fi after a password change; and resolving **SecureToken / local-account divergence** (re-aligning the local account password and SecureToken with the current Entra password after a reset).

1. Package the guidance/script for delivery via Intune: `https://intune.microsoft.com` to **Devices →

macOS → Scripts** (shell script) or a documented runbook, scoped to `MACLOCK Scope - Pilot - Macs`.

1. Keep the endpoint action **gated by change control** — remediation is authentication-impacting.

Step 6 — Pilot the remediation on the affected Macs

1. For the pilot users identified in Step 2, run the Mac-side remediation on their Macs (clear stale

keychain items, re-enter the current password in Mail/Wi-Fi, resolve SecureToken divergence).

1. Confirm the user can sign in cleanly (no repeated prompts) and that the Mac stops re-submitting the

old password.

1. File the before/after evidence into `MACLOCK Remediation Evidence Pack - Mac Credential State`.

Step 7 — Validate the reduction

1. Run `MACLOCK-Validation.ps1` to confirm Smart Lockout and the MFA policy match the desired state

in the register, and to re-read Mac-originated failures for the pilot users.

1. In the Entra sign-in logs, confirm the pilot users' **Mac-originated failures have dropped** and

no new lockouts/spray detections are firing for them.

1. Run `MACLOCK-Report.ps1` to produce the before/after exposure report and file it into the evidence

pack. Use `MACLOCK-Remediation.ps1` (default `-WhatIf`) to preview any remaining safe fixes (e.g. adding a user's Mac to the pilot scope) before applying.

Step 8 — Expand and set the review cadence

1. Once the pilot validates (failures down, no legitimate-user lockouts, MFA working), expand the MFA policy and the Mac-side remediation from `MACLOCK Scope - Pilot - Macs` to `MACLOCK Scope - Broad - Macs`, and confirm the tenant-level Smart Lockout values are right for the whole fleet.
1. Set a review cadence (e.g. weekly sign-in-log review during rollout, then monthly) and re-run `MACLOCK-Discovery.ps1`, `MACLOCK-Validation.ps1`, and `MACLOCK-Report.ps1` each cycle so new Mac-originated failures are caught and remediated before they cause lockouts.

Order of operations

`Scope groups → Assess (sign-in logs + Discovery) → Smart Lockout → MFA (CA, piloted) → Prepare Mac remediation → Pilot remediation → Validate reduction → Expand + review cadence`

5. Testing Guide — macOS Identity-Threat & Account-Lockout Hardening

Run after the pilot configuration and remediation apply (Implementation Steps 6–7) and after each review cycle.

Test 1 — Mac-originated failures identified

Do: Run `MACLOCK-Discovery.ps1` and confirm the findings register (`MACLOCK Sign-in Failure Findings - Mac-Originated - Sources`) lists the affected users, Macs, apps, and error codes read from the Entra sign-in logs. **Expected:** Every Mac-originated failure source is recorded with a user, an app, an error code, and a count.

Test 2 — Smart Lockout configured

Do: In Entra, **Protection** → **Authentication methods** → **Password protection**, confirm the lockout threshold and duration match the desired values in `MACLOCK Lockout & Spray Register - Mac Fleet - Baseline`; run `MACLOCK-Validation.ps1`. **Expected:** Threshold and duration match the register; validation reports MATCH for Smart Lockout.

Test 3 — MFA enforced for the pilot

Do: Confirm `MACLOCK CA - Require MFA - Mac Pilot` is On (or evaluated in report-only) for `MACLOCK Scope - Pilot - Macs`; have a pilot user sign in. **Expected:** The pilot user is prompted for MFA; a correct-password-only attempt is not sufficient; sign-in still succeeds for the legitimate user.

Test 4 — Mac credential state remediated

Do: For a remediated pilot Mac, confirm the stale keychain items are cleared, the saved credential (Mail / Wi-Fi / app) holds the current password, and any SecureToken / local-account divergence is resolved. **Expected:** The Mac no longer re-submits the old password; the user signs in without repeated prompts.

Test 5 — Failures reduced

Do: In the Entra sign-in logs (filtered to Status = Failure, OS = macOS), compare the pilot users' failure count against the pre-remediation baseline recorded in Step 2; run `MACLOCK-Report.ps1`. **Expected:** Mac-originated failures for the pilot users have dropped materially; no new account lockouts for those users.

Test 6 — No new lockouts / spray detections for the pilot

Do: Confirm no new account-lockout or (if licensed) password-spray / risk detections are firing for the pilot users after remediation. **Expected:** The lockouts and spray detections that were attributed to those users' Macs have stopped recurring.

Test 7 — Evidence captured

Do: Confirm `MACLOCK Remediation Evidence Pack - Mac Credential State` holds dated before/after evidence (sign-in-log counts, Smart Lockout + CA settings, per-Mac remediation) from `MACLOCK-Report.ps1`. **Expected:** Each pilot user/Mac has current, dated before/after evidence.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1–4 pass; MFA works and no legitimate pilot user is locked out
Reduction pass	Tests 5–6 pass; Mac-originated failures down and no new lockouts/spray for the pilot
Evidence pass	Test 7 passes; before/after evidence captured and dated

6. Troubleshooting Guide — macOS Identity-Threat & Account-Lockout Hardening

Use `MACLOCK-Discovery.ps1` / `MACLOCK-Validation.ps1` for sign-in-failure state and Smart Lockout / MFA posture, `MACLOCK-Report.ps1` for the before/after exposure report, and `MACLOCK-Remediation.ps1` (default `-WhatIf`) for safe fixes.

A. Failures keep coming from a Mac after the password change

Symptom	Cause	Remediation
Same user's Mac still logs failed sign-ins	Keychain / mail / Wi-Fi still holds the old password and auto-retries	Clear the stale keychain items for that account, re-enter the current password in Mail / Outlook / Wi-Fi; re-run `MACLOCK-Discovery.ps1` and confirm the failures stop

Symptom	Cause	Remediation
Failures come from a specific app (e.g. Exchange)	That app's saved credential wasn't updated	Update the credential in the app profile; keychain re-sync is per-item

B. Account keeps locking out

Symptom	Cause	Remediation
Legitimate user locked repeatedly	Smart Lockout threshold too low for the burst of Mac retries, or the Mac source not yet remediated	Remediate the Mac source first (A); then confirm the Smart Lockout threshold/duration in `MACLOCK Lockout & Spray Register - Mac Fleet - Baseline` is appropriate
Lockout behaves unexpectedly on a synced user	Hybrid account: on-prem AD lockout policy interacts with cloud Smart Lockout	Confirm the account is hybrid (Requirement 2.6); coordinate the cloud + on-prem lockout policy with the directory owner

C. Password-spray / risk detection still firing

Symptom	Cause	Remediation
Spray detection persists for a user	The Mac source is still generating failures, or another (non-Mac) source exists	Confirm from the sign-in logs (OS = macOS) whether the failures are still Mac-originated; if not, the source is elsewhere and out of this solution's scope
No risk detections visible at all	Identity Protection risk detections need Entra ID P2	Confirm licensing (Requirement 2.1); Smart Lockout + MFA still work without P2

D. MFA policy problems

Symptom	Cause	Remediation
Pilot user not prompted for MFA	Policy in report-only, wrong scope, or an exclusion applies	Confirm `MACLOCK CA - Require MFA - Mac Pilot` is On and assigned to `MACLOCK Scope - Pilot - Macs`; review the sign-in log's Conditional Access evaluation
MFA policy blocks a legitimate pilot user	Grant/condition too strict, or a needed app excluded	Narrow the policy, confirm the user has a registered MFA method, and re-test in report-only before turning On

E. SecureToken / local-account divergence

• If a user is prompted repeatedly on the Mac and keychain items keep becoming invalid after a reset,

the local account password and its **SecureToken** have likely drifted from the current Entra password. Re-align the local password and SecureToken with the current Entra password (per your Mac admin runbook), then clear and re-populate the keychain items. Keep this gated by change control — it is authentication-impacting.

F. Scope / targeting problems

• If a change reaches the wrong Macs, confirm membership of `MACLOCK Scope - Pilot - Macs` /

`MACLOCK Scope - Broad - Macs`; run `MACLOCK-Set-ScopeTargeting.ps1` to confirm the groups exist and print their object IDs, and verify the Conditional Access / Intune assignments point at them.

G. Graph / permissions problems

• If a script cannot read the sign-in logs, Smart Lockout policy, or Conditional Access, confirm the

Microsoft Graph SDK is installed and consent was granted to the read scopes (`AuditLog.Read.All`, `Directory.Read.All`, `Policy.Read.All`, `IdentityRiskEvent.Read.All`); sign in with an account holding Security Reader / Security Administrator. Re-run `MACLOCK-Discovery.ps1`.

H. When to escalate

If Mac-side remediation is complete and Smart Lockout / MFA validate correctly but a user still shows Mac-originated failures or lockouts, export the discovery and report output and raise it — the source may be a device you haven't remediated yet or a non-Mac source; open a Microsoft support case for a sign-in-log / Identity Protection data question if the logs themselves look inconsistent.

7. Best Practices — Identity protection on Macs

• **Read the sign-in logs before you touch a setting.** The Entra sign-in logs (Status = Failure, OS

= macOS) tell you exactly which users, Macs, apps, and error codes are producing the noise. Record them in `MACLOCK Sign-in Failure Findings - Mac-Originated - Sources` — fix the sources you can see, not the ones you assume.

• **Fix the cause on the Mac, not just the symptom in the cloud.** Smart Lockout and MFA protect the

account, but the failed-sign-in storm keeps coming until the **stale keychain / saved credential / SecureToken** state on the Mac is cleaned up. Do both.

• **Tune Smart Lockout for real users, not attackers.** Set the threshold high enough to tolerate a

brief burst of legitimate Mac retries but low enough to blunt a spray; record the values in `MACLOCK Lockout & Spray Register - Mac Fleet - Baseline` so the choice is deliberate and repeatable.

• **Know your lockout model before tuning.** Hybrid (AD-synced) accounts interact with an on-premises

lockout policy; coordinate cloud Smart Lockout changes with the on-prem owner so they don't fight.

• **Enforce MFA so a lucky spray hit is worthless.** A correct-password-only attempt should never be enough. Pilot the Conditional Access MFA policy (`MACLOCK CA - Require MFA - Mac Pilot`) in report-only first to avoid locking anyone out.

• **Re-sync credentials at every password change.** The classic trigger is a password change where the Mac keychain / mail profile / Wi-Fi still holds the old password. Make credential re-sync a standard post-password-change step for Mac users.

• **Keep SecureToken aligned.** After any local-account or password reset, confirm the SecureToken and local password match the current Entra password so keychain items stay valid and the Mac stops re-prompting.

• **Pilot everything that touches sign-in.** Scope MFA and remediation to `MACLOCK Scope - Pilot - Macs` first; sign-in changes are easy to break fleet-wide.

• **Least privilege for operators.** Use Security Administrator plus Conditional Access Administrator as needed; reserve Global Administrator for genuinely tenant-wide changes. This is Zero-Trust, least-privilege by default.

• **Remediate safely.** Run `MACLOCK-Remediation.ps1` in its default `-WhatIf` mode, review the proposed changes, then apply — never blind-apply an authentication-impacting change.

• **Re-validate on a cadence.** New password changes create new stale-credential situations; re-run `MACLOCK-Discovery.ps1` / `MACLOCK-Validation.ps1` / `MACLOCK-Report.ps1` on a schedule so fresh Mac-originated failures are caught before they cause the next lockout.

8. Deliverables Summary — macOS Identity-Threat & Account-Lockout Hardening

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	How macOS auth, saved credentials, keychain, and SecureToken drive failed sign-ins, lockouts, and spray noise, and the hardening model
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Security)	Category-wide prerequisites, published once

Document	Purpose
3. Implementation Guide	Exact, ordered steps: identify Mac-originated sources, enforce Smart Lockout + MFA, remediate Mac credential/keychain state, validate the reduction, expand
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Identity protection on Macs

PowerShell scripts

Script	What it does
`MACLOCK-Discovery.ps1`	Reads Entra sign-in-log failures by device/OS (macOS), user, app, and error code, plus current Smart Lockout / MFA posture (read-only)
`MACLOCK-Validation.ps1`	Confirms Smart Lockout + the MFA policy match the desired state and re-reads Mac-originated failures (read-only)
`MACLOCK-Report.ps1`	Before/after lockout-and-spray exposure report for the evidence pack (read-only)
`MACLOCK-Remediation.ps1`	Safe remediation guidance/actions for Mac-originated failures; defaults to `-WhatIf`
`MACLOCK-Set-ScopeTargeting.ps1`	Creates the Mac scope groups (pilot / broad)

Named objects (reused verbatim)

- `MACLOCK Sign-in Failure Findings - Mac-Originated - Sources`
- `MACLOCK Lockout & Spray Register - Mac Fleet - Baseline`
- `MACLOCK Remediation Evidence Pack - Mac Credential State`
- `MACLOCK CA - Require MFA - Mac Pilot` (and, if P2, `MACLOCK CA - Risk-Based MFA - Mac Pilot`)
- `MACLOCK Scope - Pilot - Macs` and `MACLOCK Scope - Broad - Macs`

What the buyer receives

- This implementation guide as a branded Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable process following the model: Assess → Configure → Pilot → Validate → Expand, on a review cadence.

Outcome

Fewer account lockouts and less password-spray noise from the Mac fleet — Mac-originated sources identified from the sign-in logs, Smart Lockout + MFA enforced, and stale Mac credential/keychain state cleaned up — proven on a pilot before you scale it across every Mac.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

MACLOCK-Discovery.ps1

Purpose: Reads Microsoft Entra sign-in logs and reports the FAILED sign-ins whose operating system is macOS, grouped by user, application, and failure/error code, so you can see which Macs are driving account lockouts and password-spray detections. Also reads the current Smart Lockout (password protection) settings and the Conditional Access / MFA policies so you can record the starting posture. Read-only. Use 'your-tenant-id' style values, not bracketed placeholders. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MACLOCK-Discovery.ps1 Optional: ... .\MACLOCK-Discovery.ps1 -Days 7 Requires the Microsoft Graph PowerShell SDK and consent to: AuditLog.Read.All, Directory.Read.All, Policy.Read.All, IdentityRiskEvent.Read.All. EXPECTED RESULTS A table of Mac-originated failed sign-ins (user, app, error code, count), a note of any password-spray / risk detections (if licensed), and the current Smart Lockout threshold / duration and MFA-enforcing Conditional Access policies — printed so you can transcribe them into 'MACLOCK Sign-in Failure Findings - Mac-Originated - Sources' and 'MACLOCK Lockout & Spray Register - Mac Fleet - Baseline'. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. Uses documented endpoints: /auditLogs/signIns, /identityProtection/riskDetections, /policies/authenticationMethodsPolicy (and legacy password-protection where surfaced), /identity/conditionalAccess/policies. #> [CmdletBinding()] param([int]$Days = 7)

How to run: powershell -ExecutionPolicy Bypass -File .\MACLOCK-Discovery.ps1 Optional: ... .\MACLOCK-Discovery.ps1 -Days 7 Requires the Microsoft Graph PowerShell SDK and consent to: AuditLog.Read.All, Directory.Read.All, Policy.Read.All, IdentityRiskEvent.Read.All. EXPECTED RESULTS A table of Mac-originated failed sign-ins (user, app, error code, count), a note of any password-spray / risk detections (if licensed), and the current Smart Lockout threshold / duration and MFA-enforcing Conditional Access policies — printed so you can transcribe them into 'MACLOCK Sign-in Failure Findings - Mac-Originated - Sources' and 'MACLOCK Lockout & Spray Register - Mac Fleet - Baseline'. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. Uses documented endpoints: /auditLogs/signIns, /identityProtection/riskDetections, /policies/authenticationMethodsPolicy (and legacy password-protection where surfaced), /identity/conditionalAccess/policies. #> [CmdletBinding()] param([int]$Days = 7)

MACLOCK-Remediation.ps1

Purpose: Works the Mac-originated failure findings from MACLOCK-Discovery.ps1 and produces the safe, well-understood remediation actions that reduce them. Two kinds of action: (1) tenant-side scoping actions this script can perform via Microsoft Graph (e.g. add an affected user to the Mac pilot scope so the MFA policy applies) and (2) endpoint actions on the Mac that are reported as MANUAL with exact steps (clear stale keychain items, re-enter the current password in Mail/Wi-Fi, resolve SecureToken / local-account divergence). SAFE BY DEFAULT: runs in -WhatIf and only reports what it WOULD do; pass -Apply to make the supported changes. It never silently rewrites a user's keychain or SecureToken. Use 'your-tenant-id' style values, not bracketed placeholders. HOW TO RUN Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\MACLOCK-Remediation.ps1 Apply after review: ... .\MACLOCK-Remediation.ps1 -Apply With your findings file: ... .\MACLOCK-Remediation.ps1 -FindingsPath .\maclock-findings.json -Apply Requires the Microsoft Graph PowerShell SDK and consent to Group.ReadWrite.All (for the pilot scoping action) — granted on connect. EXPECTED RESULTS In

preview mode: a list of the tenant-side scoping changes that WOULD be made and the MANUAL Mac-side steps to perform, with nothing altered. In -Apply mode: the same list, with each supported scoping change applied and confirmed. MANUAL steps are always reported, never attempted. .NOTES White-label — the repeatable way. SAFE BY DEFAULT (-WhatIf). Never blind-apply an authentication-impacting change; review the preview, confirm the change-control ticket and rollback, then -Apply. Endpoint keychain / SecureToken fixes are performed on the Mac per the Implementation Guide, not remotely by this script. #> [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='High')] param([string]$FindingsPath, [string]$PilotGroupName = 'MACLOCK Scope - Pilot - Macs', [switch]$Apply)

How to run: Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\MACLOCK-Remediation.ps1 Apply after review: ... .\MACLOCK-Remediation.ps1 -Apply With your findings file: ... .\MACLOCK-Remediation.ps1 -FindingsPath .\maclock-findings.json -Apply Requires the Microsoft Graph PowerShell SDK and consent to Group.ReadWrite.All (for the pilot scoping action) — granted on connect. EXPECTED RESULTS In preview mode: a list of the tenant-side scoping changes that WOULD be made and the MANUAL Mac-side steps to perform, with nothing altered. In -Apply mode: the same list, with each supported scoping change applied and confirmed. MANUAL steps are always reported, never attempted. .NOTES White-label — the repeatable way. SAFE BY DEFAULT (-WhatIf). Never blind-apply an authentication-impacting change; review the preview, confirm the change-control ticket and rollback, then -Apply. Endpoint keychain / SecureToken fixes are performed on the Mac per the Implementation Guide, not remotely by this script. #> [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='High')] param([string]$FindingsPath, [string]$PilotGroupName = 'MACLOCK Scope - Pilot - Macs', [switch]$Apply)

MACLOCK-Report.ps1

Purpose: Produces the evidence-facing report: the count and detail of macOS-originated failed sign-ins over a window (by user, app, and error code), any password-spray / risk detections (if licensed), and the current Smart Lockout / MFA posture. Exports to CSV/HTML so the output can be filed, dated, into 'MACLOCK Remediation Evidence Pack - Mac Credential State' to show the before/after reduction. Read-only. Use 'your-tenant-id' style values, not bracketed placeholders. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MACLOCK-Report.ps1 -OutputFolder .\evidence -Days 7 Requires the Microsoft Graph PowerShell SDK and consent to: AuditLog.Read.All, Policy.Read.All, IdentityRiskEvent.Read.All, Directory.Read.All. EXPECTED RESULTS A console summary plus, in the output folder, dated files: Mac-SignIn-Failures-<date>.csv (per-user/app/error failure detail) Mac-Failure-Summary-<date>.csv (counts by user) Mac-Exposure-<date>.html (readable exposure table) File these into the evidence pack, dated, to show the failure/lockout reduction. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. Uses /auditLogs/signIns, /identityProtection/riskDetections, and the password-protection / Conditional Access policy reads used by MACLOCK-Discovery.ps1. #> [CmdletBinding()] param([string]$OutputFolder = (Join-Path (Get-Location) 'evidence'), [int]$Days = 7)

How to run: powershell -ExecutionPolicy Bypass -File .\MACLOCK-Report.ps1 -OutputFolder .\evidence -Days 7 Requires the Microsoft Graph PowerShell SDK and consent to: AuditLog.Read.All, Policy.Read.All, IdentityRiskEvent.Read.All, Directory.Read.All. EXPECTED RESULTS A console summary plus, in the output folder, dated files: Mac-SignIn-Failures-<date>.csv (per-user/app/error failure detail) Mac-Failure-Summary-<date>.csv (counts by user) Mac-Exposure-<date>.html (readable exposure table) File these into the evidence pack, dated, to show the failure/lockout reduction. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. Uses /auditLogs/signIns, /identityProtection/riskDetections, and the password-protection / Conditional Access policy reads used by

MACLOCK-Discovery.ps1. #> [CmdletBinding()] param([string]$OutputFolder = (Join-Path (Get-Location) 'evidence'), [int]$Days = 7)

MACLOCK-Set-ScopeTargeting.ps1

Purpose: Creates (idempotently) the Entra security groups used to scope the macOS identity-hardening pilot and expansion, following the solution naming convention so the MFA Conditional Access policy and the Intune-delivered Mac remediation can be assigned consistently: MACLOCK Scope - Pilot - Macs MACLOCK Scope - Broad - Macs Existing groups (matched by displayName) are reused, not duplicated. Members are NOT added — add the pilot users (or their Macs) by hand or via dynamic rules per your standard. Use 'your-tenant-id' style values, not bracketed placeholders. HOW TO RUN Report only (default, safe): powershell -ExecutionPolicy Bypass -File .\MACLOCK-Set-ScopeTargeting.ps1 Apply: ... .\MACLOCK-Set-ScopeTargeting.ps1 -Apply Requires the Microsoft Graph SDK and Group.ReadWrite.All. EXPECTED RESULTS The two scope groups exist (created if missing, reused if present), with their object IDs printed for use in the MFA Conditional Access assignment (Implementation Step 4) and the Intune remediation assignment (Implementation Step 5). .NOTES White-label — the repeatable way. Safe by default (-WhatIf); pass -Apply to create groups. #> [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='Medium')] param([switch]$Apply)

How to run: Report only (default, safe): powershell -ExecutionPolicy Bypass -File .\MACLOCK-Set-ScopeTargeting.ps1 Apply: ... .\MACLOCK-Set-ScopeTargeting.ps1 -Apply Requires the Microsoft Graph SDK and Group.ReadWrite.All. EXPECTED RESULTS The two scope groups exist (created if missing, reused if present), with their object IDs printed for use in the MFA Conditional Access assignment (Implementation Step 4) and the Intune remediation assignment (Implementation Step 5). .NOTES White-label — the repeatable way. Safe by default (-WhatIf); pass -Apply to create groups. #> [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='Medium')] param([switch]$Apply)

MACLOCK-Validation.ps1

Purpose: Confirms the tenant matches the desired hardening state recorded in the register 'MACLOCK Lockout & Spray Register - Mac Fleet - Baseline': the Smart Lockout threshold/duration and an MFA-enforcing Conditional Access policy for the Mac pilot. Then re-reads the Entra sign-in logs for macOS-originated failures so you can confirm they dropped after remediation. Reports each item as MATCH or GAP. Read-only. Use 'your-tenant-id' style values, not bracketed placeholders. HOW TO RUN Default (uses bundled desired values): powershell -ExecutionPolicy Bypass -File .\MACLOCK-Validation.ps1 With your own desired state (JSON: { lockoutThreshold; lockoutDurationSeconds; mfaPolicyName }): ... .\MACLOCK-Validation.ps1 -RegisterPath .\maclock-register.json -Days 7 Requires the Microsoft Graph PowerShell SDK and consent to: AuditLog.Read.All, Policy.Read.All, Directory.Read.All. EXPECTED RESULTS A per-item MATCH / GAP list for Smart Lockout and the MFA policy, plus the current count of macOS-originated sign-in failures in the window. GAP lines are the backlog to work through in the Implementation Guide (safely via MACLOCK-Remediation.ps1). .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. The desired state is intentionally external (a JSON file you own); the bundled sample lets the script run out-of-the-box. #> [CmdletBinding()] param([string]$RegisterPath, [int]$Days = 7)

How to run: Default (uses bundled desired values): powershell -ExecutionPolicy Bypass -File .\MACLOCK-Validation.ps1 With your own desired state (JSON: { lockoutThreshold; lockoutDurationSeconds; mfaPolicyName }): ... .\MACLOCK-Validation.ps1 -RegisterPath .\maclock-register.json -Days 7 Requires the Microsoft Graph PowerShell SDK and consent to:

AuditLog.Read.All, Policy.Read.All, Directory.Read.All. EXPECTED RESULTS A per-item MATCH / GAP list for Smart Lockout and the MFA policy, plus the current count of macOS-originated sign-in failures in the window. GAP lines are the backlog to work through in the Implementation Guide (safely via MACLOCK-Remediation.ps1). .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. The desired state is intentionally external (a JSON file you own); the bundled sample lets the script run out-of-the-box. #> [CmdletBinding()] param([string]$RegisterPath, [int]$Days = 7)